

Практическая работа № 3

Использование фреймворка Laravel для разработки многостраничного веб-приложения (развёртывание на OpenServer)

Цель: Освоить базовые концепции современного PHP-фреймворка и научиться разворачивать Laravel-проект в среде OpenServer.

Задачи работы (обновлённые)

1. **Настроить окружение:** развернуть проект на базе Laravel через Composer, интегрировать его в OpenServer (создать домен, настроить виртуальные хосты).
2. **Спроектировать архитектуру:** определить маршруты, контроллеры и представления для CRUD-операций над задачами.
3. **Реализовать логику управления задачами:** создание, просмотр, обновление, удаление (CRUD). Данные хранить в сессии (без БД) для упрощения этапа обучения.
4. **Внедрить валидацию данных:** использовать встроенные механизмы Laravel для проверки полей формы.
5. **Обеспечить базовую безопасность:** реализовать защиту от CSRF-атак, корректную обработку HTTP-методов.
6. **Оформить интерфейс:** создать адаптивный шаблон с таблицей задач, статусами и формами, используя Blade-синтаксис.
7. **Протестировать функционал:** проверить все сценарии работы, убедиться в корректности валидации и безопасности форм.
8. **Подготовить отчёт:** оформить результаты в соответствии с требованиями.

Ход работы (с учётом OpenServer)

Этап 1. Подготовка окружения и структуры проекта

1. Убедитесь, что в OpenServer включена нужная версия PHP (рекомендуется PHP 8.2 или 8.3) и активирован модуль Composer.
2. В панели OpenServer перейдите в настройки доменов и создайте новый домен (например, site-lab.local). В качестве папки проекта укажите директорию, где будет лежать Laravel-проект.
3. В терминале (можно использовать встроенный в OpenServer CMD или обычный CMD с добавлением пути к PHP) выполните:
bash
ПереноситьСвернутьКопировать
composer create-project laravel/laravel site-lab
Если папка уже создана под домен, можно сразу перейти в неё и запустить установку.
4. После установки проверьте, что файл public/index.php существует.

Результат: Проект создан, домен site-lab.local настроен в OpenServer и указывает на папку проекта.

Этап 2. Настройка публичного каталога (важно для OpenServer)

В OpenServer по умолчанию корень сайта — это папка домена. Для Laravel это должна быть папка public.

Есть два пути:

1. **(Рекомендуемый для обучения)** В настройках домена в OpenServer укажите путь к папке public внутри Laravel-проекта. Тогда адрес сайта будет `http://site-lab.local/`.
2. **(Если нельзя менять настройки домена)** Создайте в корне проекта файл `.htaccess` (если используется Apache) и настройте перенаправление на public, либо используйте симлинк.

Для Apache в папке `public/.htaccess` уже есть правила. Главное — чтобы OpenServer отдавал файлы именно из папки public.

Результат: При переходе по адресу `http://site-lab.local/` отображается стартовая страница Laravel.

Этап 3. Реализация маршрутизации

В файле `routes/web.php` опишите маршруты для всех операций с задачами:

- GET / — отображение списка задач.
- POST /tasks — создание задачи.
- PUT /tasks/{id} — обновление задачи.
- DELETE /tasks/{id} — удаление задачи.

Результат: Маршруты зарегистрированы, приложение реагирует на запросы к указанным URL.

Этап 4. Создание контроллера

Создайте контроллер:

```
bash
```

```
ПереноситьСвернутьКопировать
```

```
php artisan make:controller TaskController
```

Реализуйте в контроллере методы `index`, `store`, `update`, `destroy`. В качестве хранилища используйте сессию (`Session::get`, `Session::put`).

Результат: Логика CRUD инкапсулирована в контроллере, данные сохраняются и извлекаются из сессии.

Этап 5. Валидация данных

В методах `store` и `update` используйте `$request->validate()` для проверки:

- title: обязательное поле, длина от 3 до 200 символов.
- description: необязательное, максимум 500 символов.
- status: только значения `new`, `in_progress`, `done`.

При ошибке валидации фреймворк автоматически вернёт пользователя на предыдущую страницу с сообщениями об ошибках.

Результат: Формы не принимают некорректные данные, пользователь видит понятные сообщения об ошибках.

Этап 6. Создание представления (шаблона)

Создайте файл `resources/views/tasks/index.blade.php`. Реализуйте:

- Форму добавления задачи с CSRF-токеном (`@csrf`).
- Таблицу для отображения списка задач с циклом `@foreach`.
- Кнопки/формы для удаления (с `@method('DELETE')`) и обновления.
- Вывод сообщений об успехе (через `session('success')`).
- Стили для статусов задач (CSS-классы `.status-new`, `.status-in_progress`, `.status-done`).

Результат: Интерфейс приложения оформлен, все элементы работают, Blade-синтаксис используется корректно.

Этап 7. Тестирование и проверка безопасности (с учётом OpenServer)

Проверьте следующие сценарии:

1. **Успешное создание:** введите корректные данные, убедитесь, что задача появилась в списке.
2. **Ошибка валидации:** оставьте поле «Название» пустым, убедитесь, что форма не отправляется и выводится сообщение об ошибке.
3. **Удаление и обновление:** проверьте работу кнопок, убедитесь, что данные обновляются корректно.
4. **Защита от CSRF:** попробуйте отправить POST-запрос без токена (например, через curl или Postman) — сервер должен отклонить запрос.
5. **Эмуляция методов:** убедитесь, что формы с `@method('DELETE')` корректно обрабатываются контроллером.
6. **Проверка домена OpenServer:** убедитесь, что сайт открывается именно по домену `site-lab.local`, а не через `localhost:8000` (это подтверждает, что проект развёрнут в среде OpenServer).

Результат: Все сценарии протестированы, функционал работает стабильно, механизмы безопасности подтверждены, проект корректно работает в OpenServer.

Требования к отчёту (обновлённые)

Отчёт должен содержать:

1. **Титульный лист** (тема, дисциплина, группа, ФИО студента и преподавателя).
2. **Цель и задачи работы.**
3. **Описание структуры проекта:** перечень ключевых файлов и их назначение.
4. **Листинги ключевых фрагментов кода:** маршруты, метод `store` с валидацией, фрагмент шаблона.
5. **Скриншоты интерфейса:** главная страница, форма с ошибкой валидации, сообщение об успехе.
6. **Скриншот панели OpenServer:** на нём должно быть видно, что домен `site-lab.local` активен и указывает на правильную папку проекта. Это подтверждает выполнение требования по развёртыванию.

7. **Описание результатов тестирования:** подтверждение работы всех CRUD-операций, валидации и механизмов безопасности.
8. **Вывод:** о достижении цели работы, приобретённых навыках и перспективах дальнейшего развития проекта.
9. **Архив с полным кодом проекта** (папка проекта без папки vendor).

Критерии оценки (с учётом OpenServer)

Критерий	Баллы
Настройка окружения и интеграция с OpenServer (домен, PHP, Composer)	15
Реализация маршрутизации и контроллера	20
Валидация данных и обработка ошибок	20
Шаблонизация (Blade) и оформление интерфейса	20
Тестирование и подтверждение безопасности (CSRF, методы)	15
Оформление отчёта и архива (включая скриншот OpenServer)	10
Итого	100

Контрольные вопросы для защиты работы (добавлены про OpenServer)

1. Как вы настроили домен в OpenServer для Laravel-проекта? Почему важно, чтобы корень сайта указывал на папку public?
2. В чём отличие запуска проекта через php artisan serve и через OpenServer? Какие преимущества и недостатки у каждого подхода?
3. Как проверить, какая версия PHP используется в OpenServer для вашего проекта? Почему это важно?
4. Что такое Composer и как он используется в связке с OpenServer?
5. Остальные вопросы из предыдущей версии (про маршрутизацию, валидацию, CSRF и т. д.) остаются актуальными.

Дополнительные задания (по выбору)

- **Подключить базу данных:** заменить хранение в сессии на использование Eloquent ORM и миграции. В OpenServer можно использовать встроенную MySQL.
- **Реализовать аутентификацию:** добавить регистрацию и вход пользователей.
- **Создать API:** добавить маршруты для получения списка задач в формате JSON.
- **Добавить пагинацию:** вывести задачи постранично (по 10 записей на странице).
- **Настроить логирование:** проверить, куда Laravel пишет логи в среде OpenServer, и убедиться, что ошибки фиксируются.

Вывод

В ходе выполнения практического задания было создано многостраничное веб-приложение на фреймворке Laravel, развёрнутое в среде **OpenServer**. Студент освоил ключевые концепции фреймворка, а также научился интегрировать Laravel-проект в локальный серверный комплекс, настраивать домены и работать с версиями PHP. Полученные навыки позволяют переходить к разработке более сложных приложений с полноценной базой данных и системой аутентификации, а также уверенно работать в лабораторных условиях на базе OpenServer.

Если скажете, нужно ли добавить инструкцию по настройке MySQL в OpenServer именно для этого проекта (чтобы заменить сессию на реальную БД), я дополню ход работы отдельным этапом.